Міністерство освіти і науки України
Полтавський національний педагогічний університет
імені В. Г. Короленка
Факультет комп'ютерних наук, математики, фізики та економіки
Кафедра математичного аналізу та інформатики

Рекомендовано до захисту
«____» _____ 2026 р.,
протокол № _____
Завкафедри _____
(підпис)

Кваліфікаційна робота
на здобуття другого (магістерського)
рівня вищої освіти

«Розробка інтелектуальної EdTech-системи адаптивного навчання та децентралізованого оцінювання знань на основі моделі Bayesian Knowledge Tracing та технології блокчейн»

Виконав: студент VI курсу
денної форми навчання
спеціальності 122 Комп'ютерні науки
освітньої програми Комп'ютерні науки
Стеценко Дмитро

Науковий керівник:
_____
(науковий ступінь, вчене звання, ПІБ)

Рецензент:
_____
(науковий ступінь, вчене звання, ПІБ)

Захищено «____» _____ 2026 р.
кількість балів: _____
значення оцінки: _____
Голова Екзаменаційної комісії _____
(підпис)

Полтава – 2026

ЗМІСТ

ВСТУП

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА НАУКОВО-МЕТОДИЧНИХ ПІДХОДІВ ДО ОЦІНЮВАННЯ ЗНАНЬ

1.1 Еволюція та архітектурні обмеження систем електронного навчання (LMS, ITS, Blockchain-реєстри)

1.2 Теоретико-ймовірнісний базис моделей відстеження знань (CTT, IRT, класичний BKT, DKT)

1.3 Феноменологія когнітивних пасток та ефекту «скляної стелі» чистого BKT (Lucky Guess, Zero-Learning Sink, Frustration Loop)

1.4 Децентралізовані стандарти непередаваних цифрових активів (Soulbound Tokens) та постановка завдань розробки

РОЗДІЛ 2. МАТЕМАТИЧНЕ МОДЕЛЮВАННЯ ТА АЛГОРИТМІЗАЦІЯ АДАПТИВНОГО НАВЧАННЯ

2.1 Модифікована модель BKT із динамічними регуляторами ймовірностей P(G_t) та P(S_t)

2.2 Математичне правило числових границь з гарантованою асимптотою (Clamping Rule)

2.3 Алгоритми багатовимірної сенсорної телеметрії кодописання та інтегральний коефіцієнт довіри C_trust

2.4 Алгоритм тактичного відступу за графом пререквізитів (DAG Backtracking) та трирівневе дидактичне риштування

2.5 Математична та криптографічна модель аудиту навчальних сесій і верифікації сертифікатів

РОЗДІЛ 3. АРХІТЕКТУРА ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІНТЕЛЕКТУАЛЬНОЇ EDTECH-СИСТЕМИ

3.1 Загальна мікросервісна та мікрофронтендна архітектура монорепозиторію

3.2 Гексагональне аналітичне ядро бекенду (apps/api), WebSocket-шлюзи та схема бази даних

3.3 Клієнтські підсистеми практичного кодування та методичного керування (apps/student, apps/teacher, apps/admin)

3.4 Архітектура навчального простору первинної дидактики та рушія фокусування (apps/study та Zen Focus Reading Engine)

3.5 Смарт-контракт децентралізованої сертифікації SoulboundCertificate у мережі Polygon

РОЗДІЛ 4. ОРГАНІЗАЦІЯ ПРОЦЕСУ РОЗРОБКИ ТА УПРАВЛІННЯ ВИМОГАМИ ДО ПРОГРАМНОГО КОМПЛЕКСУ

4.1 Гнучка методологія розробки, архітектура компонентів та декомпозиція епіків у системі Jira

4.2 Регламент контролю версій, семантичне маркування коммітів (Jira Conventional Commits) та автоматизовані Git Hooks

4.3 Автоматизований експорт та синхронізація артефактів проєктної аналітики

4.4 Критерії готовності функціоналу (Definition of Done) та інженерна верифікація мікро-кроків

РОЗДІЛ 5. ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ ТА ОЦІНКА ЕФЕКТИВНОСТІ РОЗРОБЛЕНОЇ СИСТЕМИ

5.1 Методика організації комп'ютерного експерименту та характеристика тестового датасету

5.2 Емпіричне порівняння діагностичної точності моделей (RMSE, AUC-ROC, час збіжності)

5.3 Дослідження стійкості системи до академічного шахрайства та використання LLM-генераторів

5.4 Навантажувальне тестування веб-сокетного ядра та оптимізація транзакційних витрат у блокчейні

ЗАГАЛЬНІ ВИСНОВКИ

ВСТУП

Актуальність теми. Розвиток дистанційного та змішаного навчання висуває нові вимоги до засобів діагностики навчальних досягнень здобувачів вищої та профільної комп'ютерної освіти. Широке поширення генеративних моделей штучного інтелекту уможливлює швидку генерацію фрагментів програмного коду, що створює додаткові виклики для класичного підсумкового тестування без контролю динаміки написання коду. Водночас сучасні адаптивні платформи переважно орієнтовані на констатацію поточного результату, потребуючи розширення функцій своєчасного дидактичного супроводу здобувачів при виникненні труднощів. Розробка повнофункціональної інтелектуальної EdTech-системи, що органічно інтегрує адаптивне навчання на базі модифікованої моделі Bayesian Knowledge Tracing (BKT), потокову темпоральну телеметрію та децентралізовану верифікацію компетентностей на основі технологій розподіленого реєстру, становить актуальне науково-прикладне завдання комп'ютерних наук.

Трансформація парадигми вищої та профільної комп'ютерної освіти в умовах цифровізації супроводжується якісними змінами у формах представлення навчального матеріалу, організації самостійної роботи та методах оцінювання навчальних досягнень здобувачів. Широке впровадження систем електронного навчання (Learning Management Systems — LMS) та поява високопродуктивних генеративних моделей штучного інтелекту (Large Language Models — LLM: ChatGPT, Claude, GitHub Copilot) актуалізували питання підвищення об'єктивності, комплексності та академічної доброчесності діагностики практичних навичок програмування.

Традиційні підходи комп'ютеризованого тестування, що базуються на класичній теорії тестів (Classical Test Theory — CTT) та завданнях множинного вибору (Multiple Choice Questions — MCQ), успішно вирішують завдання швидкого вхідного або проміжного скринінгу теоретичних понять. Водночас у сфері комп'ютерних наук та програмної інженерії їхні діагностичні можливості

мають певні обмеження: оцінювання спирається виключно на фінальний результат виконання завдання, залишаючи поза аналізом динаміку написання та структуру побудови коду. В умовах відкритого доступу до генеративних асистентів здобувач освіти може отримати сторонній фрагмент коду та розмістити його в середовищі виконання без поглибленого аналізу алгоритму. У стандартній системі керування навчанням такий розв'язок успішно проходить модульні тести, що ускладнює своєчасну діагностику реального рівня розуміння студента та вимагає впровадження додаткових методів верифікації навчальних дій.

З іншого боку, наявні системи комп'ютеризованого адаптивного тестування (Computerized Adaptive Testing — CAT) та інтелектуальні навчальні системи (Intelligent Tutoring Systems — ITS) ефективно класифікують прогалини у знаннях, проте потребують якісного розширення контекстного дидактичного супроводу в момент виникнення складнощів. За відсутності гнучкого ступеневого підказування багаторазові повторні помилки можуть призводити до когнітивної втоми здобувача та зниження навчальної мотивації.

Для розв'язання окреслених завдань доцільним є створення інтегрованого комплексу програмних засобів — системи адаптивного навчання та об'єктивного оцінювання знань. Базовим діагностичним апаратом такої платформи обрано теорію байєсівського моделювання знань (Bayesian Knowledge Tracing — BKT), яка інтерпретує процес формування навички як прихований марковський процес. Водночас класична модель BKT Корбетта-Андерсона орієнтована на бінарні результати спроб і не враховує часову динаміку розв'язання, що зумовлює чутливість оцінки до випадкового вгадування (Lucky Guess), копіювання коду та випадкових механічних описок (Slip).

Інтеграція в модель BKT динамічних регуляторів на основі багатовимірної сенсорної телеметрії кодописання (темпу набору WPM, частки вставки з буфера обміну, міжсимвольних пауз та динаміки правок) дозволяє суттєво підвищити точність оцінювання когнітивного стану та забезпечити

об'єктивну верифікацію самостійності виконання завдань. Поєднання цього аналітичного ядра з технологіями розподіленого реєстру (блокчейн) та смарт-контрактами стандарту Soulbound Tokens (SBT, ERC-5192) забезпечує надійний механізм незмінної, публічно верифікованої та персоніфікованої фіксації освітніх досягнень здобувачів без ризику несанкціонованої передачі прав на сертифікат.

Отже, розробка теоретичних моделей, математичних методів, алгоритмів та програмних засобів інтелектуальної EdTech-системи, що органічно поєднує сенсорний ВКТ-рушій та децентралізовану сертифікацію, є актуальним науково-прикладним завданням комп'ютерних наук.

Зв'язок роботи з науковими програмами, планами, темами. Дослідження виконано відповідно до пріоритетних напрямів наукових досліджень кафедри математичного аналізу та інформатики факультету комп'ютерних наук, математики, фізики та економіки Полтавського національного педагогічного університету імені В. Г. Короленка в межах комплексної наукової теми щодо створення інтелектуальних автоматизованих систем навчання, адаптивного тестування та інтелектуальної обробки даних у відкритих освітніх середовищах.

Мета і завдання дослідження. Метою кваліфікаційної магістерської роботи є підвищення ефективності індивідуального опанування програмування та забезпечення достовірності й академічної доброчесності оцінювання знань шляхом розробки повнофункціональної інтелектуальної EdTech-системи з адаптивним навчальним супроводом, телеметричною верифікацією самостійності дій та децентралізованим карбуванням непередаваних сертифікатів (Soulbound Tokens).

Для досягнення поставленої мети сформульовано та розв'язано такі завдання:

1. Здійснити всебічний порівняльний аналіз наявних систем електронного навчання (LMS, ITS, блокчейн-реєстрів), виявити їхні архітектурні межі застосовності та математичні сліпі зони;
2. Дослідити теоретико-ймовірнісний апарат моделей відстеження знань (ВКТ,

IRT, DKT), формалізувати системні вади класичного BKT («Lucky Guess», «Zero-Learning Sink», «The Frustration Loop») та розробити модифіковану модель із динамічними регуляторами на основі темпоральної телеметрії;

3. Спроєктувати та реалізувати архітектуру інтелектуальної EdTech-екосистеми з чітким рольовим розподілом мікрофронтендів (apps/study, apps/student, apps/teacher, apps/admin) та гексагонального аналітичного ядра (apps/api);

4. Розробити математичні алгоритми та протоколи надійного функціонування клієнтського сенсорного пайплайну телеметрії, захисту від повторних сабмітів та Human-in-the-Loop дедуплікації черги перевірки;

5. Спроєктувати комплекс децентралізованої фіксації компетентностей на базі смарт-контрактів стандарту Soulbound Tokens (ERC-5192) із захистом від транзакційного перевантаження мережі;

6. Організувати інженерний процес розробки та управління вимогами за методологією Agile/Jira, формалізувати регламент семантичних коммітів Jira Conventional Commits та критерії готовності коду (Definition of Done);

7. Провести масштабне емпіричне експериментальне дослідження точності модифікованої моделі BKT (RMSE, AUC-ROC), стійкості до списування та продуктивності компонентів системи.

Об'єкт дослідження — Процес організації адаптивного комп'ютеризованого навчання, контролю знань та децентралізованої сертифікації компетентностей здобувачів освіти.

Предмет дослідження — Методи та математичні моделі адаптивного оцінювання знань на базі Bayesian Knowledge Tracing, алгоритми аналізу темпоральної поведінкової телеметрії взаємодії користувача з кодовим середовищем, компонентна мікрофронтендна архітектура EdTech-платформи та смарт-контракти розподіленої верифікації компетентностей.

Методи дослідження. Для розв'язання поставлених завдань використано:

- Теорія ймовірностей та математична статистика, теорія прихованих марковських моделей — для побудови та математичного обґрунтування адаптивного BKT-рушія;

- Теорія графів та відношень порядку — для проєктування структури навчальних пререквізитів та алгоритму тактичного обходу графа DAG Backtracking;
- Методи системного аналізу, об'єктно-орієнтованого та архітектурного проєктування програмних систем — для створення мікросервісної та мікрофронтендної структури монорепозиторію;
- Методи комп'ютерного моделювання, емпіричного тестування та статистичного аналізу — для експериментальної верифікації метрологічних і швидкісних характеристик системи.

Наукова новизна одержаних результатів:

1. Дістала подальший розвиток модель Bayesian Knowledge Tracing завдяки введенню динамічних коригувальних регуляторів ймовірностей вгадування P(G_t) та помилки P(S_t), які вперше розраховуються на основі багатовимірного темпорального аналізу потоку взаємодії користувача з кодовим редактором (WPM, коефіцієнт вставки, латентність першого натискання, динаміка правок). Це дозволило зменшити середньоквадратичну похибку діагностики знань на 44% порівняно зі стандартною моделлю Корбетта-Андерсона.
2. Удосконалено алгоритм адаптивної ремедіації (Remediation Scaffolding) через інтеграцію тактичного відступу за графом передумов (DAG Backtracking) та карток ментальних оман (Misconceptions), що запобігає виникненню небажаного зациклення при повторних помилках здобувача освіти.
3. Запропоновано науково-обґрунтовану гібридну дворівневу архітектуру децентралізованої фіксації навчальних досягнень, де динамічні обчислення виконуються в оптимізованому off-chain середовищі, а в розподілений реєстр карбуються непередавані сертифікати Soulbound Tokens (ERC-5192) з криптографічним аудиторським хешем сесії.

Практичне значення одержаних результатів:

Розроблено та практично впроваджено повнофункціональний програмний комплекс інтелектуальної EdTech-системи, що включає:

- Портал дидактичного вивчення теорії та ремедіації apps/study з авторським

рушієм Zen Focus Reading Engine та інтерактивною пісочницею CodePlayground;

- Інтерактивний тренажер кодування apps/student з інтеграцією Monaco Editor, сенсорним перехоплювачем дій useBehavioralTelemetry та візуалізацією індивідуального графу знань;
- Аналітичне робоче місце викладача apps/teacher з когортною тепловою картою BKT Heatmap, радаром виявлення аномалій та інтерактивною чергою верифікації ReviewQueueInspector;
- Адміністративний центр методиста apps/admin для візуального конструювання навчального графу та налаштування параметрів завдань;
- Гексагональне аналітичне ядро apps/api на NestJS з підтримкою високошвидкісних WebSocket-комунікацій;
- Комплект смарт-контрактів SoulboundCertificate.sol для блокчейн-мережі другого рівня Polygon.

Апробація результатів дослідження. Основні теоретичні та прикладні положення кваліфікаційної магістерської роботи доповідалися, обговорювалися та отримали схвальну оцінку на науково-практичних семінарах кафедри математичного аналізу та інформатики факультету комп'ютерних наук, математики, фізики та економіки ПНПУ імені В. Г. Короленка (Полтава, 2026 р.).

Публікації. За результатами проведених досліджень підготовлено наукову публікацію для подання до фахового наукового видання України категорії «Б» з комп'ютерних наук та програмної інженерії.

Структура та обсяг роботи. Кваліфікаційна робота складається зі вступу, п'яти взаємопов'язаних розділів, загальних висновків та додатків. Загальний обсяг роботи становить 65 сторінок комп'ютерного тексту, містить 13 аналітичних таблиць, 28 математичних формул та 5 лістингів програмного коду.

РОЗДІЛ 1
АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА НАУКОВО-МЕТОДИЧНИХ ПІДХОДІВ ДО ОЦІНЮВАННЯ ЗНАНЬ

1.1. Еволюція та архітектурні обмеження систем електронного навчання (LMS, ITS, Blockchain-реєстри)

1.1. Еволюція та архітектурні особливості систем електронного навчання (LMS, ITS, Blockchain-реєстри)

Розвиток дистанційних та змішаних освітніх технологій в університетах актуалізував завдання об'єктивного оцінювання практичних умінь здобувачів вищої освіти з програмування в умовах опосередкованої взаємодії через вебінтерфейс. Поява загальнодоступних великих мовних моделей (ChatGPT, Claude, GitHub Copilot) створила нові дидактичні умови: наявність інструментів автоматичної кодогенерації дозволяє швидко отримувати синтаксично коректні фрагменти програм, що вимагає оновлення підходів до діагностики реального розуміння алгоритмів здобувачами.

Для визначення передумов побудови інтегрованої системи доцільно проаналізувати функціональні характеристики трьох класів освітнього програмного забезпечення.

По-перше, це платформи керування навчанням (LMS: Moodle, Canvas LMS, Blackboard, Open edX).

Зазначені системи успішно виконують функції централізованого навчального менеджменту: адміністрування облікових записів, планування розкладу, структуроване представлення навчальних ресурсів та ведення електронних відомостей. Діагностичний апарат базового тестування в них традиційно спирається на положення класичної теорії тестів (Classical Test Theory, CTT).

В основі CTT лежить модель адитивної декомпозиції балу:

$$X = T + E \tag{1.1}$$

де спостережуваний результат X інтерпретується як сума справжнього значення латентної властивості T та випадкової похибки вимірювання E. Практична реалізація методу полягає у підсумовуванні балів за успішно пройдені контрольні завдання.

При використанні спеціалізованих модулів оцінювання програмного коду (зокрема VPL — Virtual Programming Lab або CodeRunner) перевірка організована за принципом замкненого середовища:

1. Здобувач завантажує вихідний файл або вводить код у вебформу;
2. Серверна підсистема ініціалізує ізольований контейнер Docker або середовище компіляції;
3. Програма верифікується набором модульних тестів (перевірка потоків введення/виведення STDIN/STDOUT);
4. Підсумковий результат фіксується у журналі оцінок.

Така організація забезпечує надійну перевірку синтаксичної та функціональної коректності програми, проте вона зосереджена виключно на фінальному статичному стані файлу. Процес конструювання коду — темпоральні інтервали між правками, структура введення, наявність тривалих періодів обдумування чи миттєве завантаження готових блоків — залишається поза межами аналітичного контуру класичної LMS. Крім того, централізоване використання контейнеризованих середовищ перевірки вимагає масштабованих обчислювальних ресурсів під час пікових навантажень при синхронній роботі студентських груп.

По-друге, це адаптивні системи та інтелектуальні навчальні системи (ITS: ALEKS, Knewton Alta, Carnegie Learning).

Розробка систем цього класу ґрунтується на розгалуженому математичному апараті: теорії просторів знань (Knowledge Space Theory, KST) та теорії тестових завдань (Item Response Theory, IRT). У межах KST навчальні концепції структуруються як сітка взаємозумовлених станів знання, тоді як IRT забезпечує калібрування параметрів завдань за рівнем складності та дискримінативності.

Водночас у межах класичних ITS діагностичний фокус здебільшого зосереджений на ідентифікації прогалин у знаннях. Якщо учень припускається повторних концептуальних помилок (наприклад, у темах покажчиків чи рекурсивних викликів), система зазвичай генерує додаткові завдання спорідненого типу. За відсутності ступеневої підказки та адаптивного переходу до базових пререквізитів серія невдач може спричиняти зниження мотивації студента. Опитування та аналіз навчальної активності свідчать, що у подібних ситуаціях до 42% здобувачів припиняють виконання модуля без завершення. Крім того, комерційні платформи функціонують переважно як закриті екосистеми, що ускладнює експорт первинних інженерних логів для зовнішньої незалежної валідації.

По-третє, це блокчейн-реєстри академічних сертифікатів (MIT Blockcerts, OpenCerts).

Використання розподілених реєстрів забезпечує криптографічну цілісність підсумкових документів про освіту: хеш-сума сертифіката засвідчується цифровим підписом закладу вищої освіти та записується в незмінну мережу.

Проте блокчейн-реєстри фіксують виключно фінальний результат атестації та за своєю природою не здійснюють моніторинг самого процесу формування компетентностей. У разі застосування звичайних токенів стандарту ERC-721 або ERC-1155 також виникає потенційний ризик передачі або перереєстрації цифрового активу між різними обліковими записами, що суперечить принципу персоніфікованої атестації.

Порівняльний аналіз зазначених технологічних підходів наведено в таблиці 1.1.

Таблиця 1.1 — Порівняльний аналіз архітектурних характеристик освітніх систем

Платформа / Система	Клас архітектури	Діагностичний апарат	Затримка реакції системи	Контроль динаміки написання коду
Moodle / Canvas LMS	Корпоративна LMS	Класична теорія тестів (CTT)	Дискретна (після завершення)	Відсутній (фіксація статичного сабміту)
ALEKS (McGraw-Hill)	Адаптивна ITS	Теорія просторів знань (KST)	1500 - 3000 мс (хмарний HTTP)	Частковий (фіксація сумарного часу сесії)
Knewton Alta	Адаптивна ITS	Item Response Theory (IRT/BKT)	800 - 1500 мс (REST API)	Опосередкований (контроль тривалості кроку)
MIT Blockcerts	Блокчейн-реєстр	Відсутній (статичний хеш документа)	15 - 60 с (підтвердження блоку)	Відсутній (верифікація фінального сертифіката)
Smart-BKT-Chain	Інтелектуальна EdTech	Модифікований BKT + Телеметрія	< 5 мс (WebSocket ядро)	Комплексний (багатовимірна темпоральна телеметрія).

Порівняльний аналіз демонструє, що сучасні платформи електронного навчання успішно вирішують поставлені перед ними функціональні завдання у межах власної спеціалізації (адміністрування контенту, адаптивне тестування, незмінне збереження дипломів). Проте в умовах масового поширення засобів генеративного кодописання виникає об'єктивна науково-прикладна потреба у створенні цілісної інтегрованої EdTech-системи. У такій системі процес оцінювання органічно доповнюється багатовимірною телеметрією кодописання, алгоритмічний апарат своєчасно надає адаптивну дидактичну підтримку під час виникнення труднощів, а підсумкові компетентності фіксуються децентралізованими цифровими свідоцтвами.

1.2. Теоретико-ймовірнісний базис моделей відстеження знань (CTT, IRT, класичний BKT, DKT)

1.2. Теоретико-ймовірнісний базис моделей відстеження знань (CTT, IRT, класичний BKT, DKT)

Під терміном «відстеження знань» (Knowledge Tracing, KT) у комп'ютерних науках розуміють динамічне моделювання латентного (прихованого) когнітивного стану учня. Ми не можемо зазирнути в голову студента, але спостерігаємо послідовність його дій — правильні рішення, помилки, затримки між рядками та виправлення.

За останні десятиліття в освітньому дата-майнінгу сформувалося кілька конкуруючих математичних шкіл.

Перший напрям — моделі сімейства Item Response Theory (IRT).

У трипараметричній версії 3PL IRT ймовірність того, що учень із рівнем здатності θ розв'яже задачу j, задається логістичною кривою:

$$P(Y_{ij} = 1 \mid \theta_i) = c_j + (1 - c_j / 1 + exp(-a_j (\theta_i - b_j))) \quad (1.2)$$

де параметр a_j задає дискримінаційну крутизну завдання, b_j — його складність, а c_j — імовірність вгадування.

Головне обмеження IRT полягає в тому, що здатність θ розглядається як константа протягом випробування. Модель добре підходить для зрізового тестування (на кшталт ЗНО чи SAT), але вона не розрахована на ситуацію, коли учень вчиться прямо під час виконання лабораторної роботи. До того ж оцінювання параметрів методом максимальної правдоподібності (MML) вимагає чисельних ітерацій, які важко проводити в реальному часі для сотень користувачів одночасно.

Другий напрям — байєсівське відстеження знань (Bayesian Knowledge Tracing, BKT).

Модель, розроблена Альбертом Корбеттом та Джоном Андерсоном у 1994 році, стала класикою адаптивного навчання. Вона розглядає засвоєння кожної навички як дискретний марковський процес із двома прихованими станами: 0

— навичка ще не засвоєна, 1 — навичка засвоєна.

Учень взаємодіє з системою покроково. На кожному кроці t спостерігається бінарний результат: правильна відповідь (Y_t = 1) або помилка (Y_t = 0).

Поведінка моделі задається чотирма параметрами:

- P(L_0) — початкова ймовірність знання до початку роботи;
- P(T) — ймовірність переходу зі стану 0 у стан 1 (ймовірність навчитися завдяки поточній спробі);
- P(G) — ймовірність випадкового вгадування (Guess): учень не знав правила, але відповідь зійшлася;
- P(S) — ймовірність прикрого промаху (Slip): учень знав матеріал, але припустився механічної описки.

Перерахунок стану знань виконується у два такти.

Спочатку оновлюємо ймовірність за правилом Байєса:

Якщо результат успішний (Y_t = 1):

$$P(L_t \mid Y_t = 1) = (P(L_{t-1}) \cdot (1 - P(S)) / P(L_{t-1}) \cdot (1 - P(S)) + (1 - P(L_{t-1})) \cdot P(G)) \quad (1.3)$$

Якщо сталася помилка (Y_t = 0):

$$P(L_t \mid Y_t = 0) = (P(L_{t-1}) \cdot P(S) / P(L_{t-1}) \cdot P(S) + (1 - P(L_{t-1})) \cdot (1 - P(G))) \quad (1.4)$$

Потім додаємо ймовірність того, що студент засвоїв матеріал у процесі поточної спроби:

$$P(L_{t+1}) = P(L_t \mid Y_t) + (1 - P(L_t \mid Y_t)) \cdot P(T) \quad (1.5)$$

Величезна перевага BKT — повна прозорість і швидкість. Кожен крок прораховується за частки мікросекунди без складних матричних множень, а значення P(L) легко інтерпретується викладачем як ступінь упевненості від 0 до 100%.

Третій напрям — нейромережевий Deep Knowledge Tracing (DKT).

У 2015 році Кріс Пєх запропонував відмовитися від ручних марковських моделей на користь рекурентних нейромереж (LSTM):

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h), \quad y_t = \sigma(W_{hy} h_t + b_y) \qquad (1.6)$$

Модель DKT добре показує себе на гігантських датасетах (ASSISTments, EdNet), де є мільйони записів. Але для реального університетського курсу вона створює більше проблем, ніж розв'язує:

- Приховані ваги h_t неможливо пояснити: викладач не розуміє, чому одному студенту система зарахувала тему, а іншому ні;
- Нейромережа схильна до перенавчання на малих вибірках (кілька десятків студентів академічної групи);
- Прямий прохід через важку модель створює непотрібне навантаження на сервер під час практичних занять.

Зіставлення моделей зведено в таблиці 1.2.

Таблиця 1.2 — Порівняльний аналіз математичних моделей відстеження знань

Критерій порівняння	**Модель IRT (1PL / 3PL)**	**Класична модель BKT**	**Нейромережева модель DKT**	**Запропонована Telemetry-BKT**
Врахування приросту знань	Відсутнє (параметр здатності константний)	Присутнє (через імовірність переходу P(T))	Присутнє (через зміну прихованого вектора h_t)	Присутнє (байєсівське динамічне моделювання)
Педагогічна інтерпретованість	Середня (шкала логітів здатності)	Висока (ймовірність знання $P(L) \in [0; 1]$)	Вкрай низька (неінтерпретовані приховані ваги)	Абсолютна (детальні коефіцієнти довіри C_trust)
Обчислювальна складність	Висока (ітеративні чисельні методи)	Мінімальна (аналітичні прямі формули)	Дуже висока (тензорні операції GPU/CPU)	Мінімальна (< 1.2 мс на один крок розрахунку)
Чутливість до процесу вводу	Відсутня (лише бінарний результат)	Відсутня у класичній версії	Опосередкована (вимагає складних ембедінгів)	Пряма (реалтайм регулятори за телеметрією).

Зрозуміло, що BKT залишається оптимальним каркасом. Але його класичний вигляд вимагає глибокої переробки під реалії сучасного кодописання.

1.3. Феноменологія когнітивних пасток та ефекту «скляної стелі» чистого BKT (Lucky Guess, Zero-Learning Sink, Frustration Loop)

1.3. Граничні математичні режими та обмеження класичної моделі BKT (Lucky Guess, Zero-Learning Sink, Frustration Loop)

Детальний емпіричний аналіз показав, що при безпосередньому застосуванні канонічних формул Корбетта-Андерсона до практичних завдань з алгоритмізації та програмування виникає низка специфічних математичних крайових ефектів. У межах дослідження виділено три ключові обмеження класичного ймовірнісного апарату, які утворюють ефект асимптотичного насичення оцінки чистого BKT в інтерактивному середовищі кодописання.

Особливість 1. Некоректне зростання оцінки при сторонньому введенні («Lucky Guess Spike»).

У класичній формулі параметри P(G) та P(S) розглядаються як незмінні константи (зазвичай приймають P(G) = 0.20, P(S) = 0.10).

Розглянемо випадок, коли здобувач знаходиться на початковому етапі засвоєння теми («Рекурсивні структури даних»), і його поточна ймовірність знання становить P(L) = 0.15. Учень завантажує сторонній згенерований розв'язок у поле редактора та ініціює модульне тестування, яке повертає позитивний результат (Y_t = 1).

Розрахунок за класичною формулою (1.3):

$$P(L_t \mid Y_t = 1) = (0.15 \cdot 0.90 / 0.15 \cdot 0.90 + 0.85 \cdot 0.20) = (0.135 / 0.135 + 0.170) \approx 0.443 \quad (1.7)$$

З урахуванням складової навчання P(T) = 0.20 за формулою (1.5):

$$P(L_{t+1}) = 0.443 + (1 - 0.443) \cdot 0.20 = 0.554 \quad (1.8)$$

При наступній аналогічній швидкій вставці значення P(L) перевищує 0.85, і система формально фіксує досягнення майстерності. Статична модель не враховує часові обмеження людини: за короткий темпоральний інтервал

здобувач об'єктивно не може самостійно спроєктувати та набрати алгоритмічний код відповідного рівня складності.

Особливість 2. Математичне згасання та інертність реабілітації («Zero-Learning Sink»).

Інша ситуація виникає при серії механічних синтаксичних помилок сумлінних здобувачів. Якщо студент послідовно припускається дрібних неточностей (наприклад, 5 невдалих спроб компіляції поспіль):

Після першої невдачі при початковому P(L) = 0.50:

$$P(L_1) = (0.50 \cdot 0.10 / 0.50 \cdot 0.10 + 0.50 \cdot 0.80) = (0.05 / 0.45) \approx 0.111 \qquad (1.9)$$

Після п'ятої помилки розрахункова оцінка знань асимптотично наближається до нуля (< 0.0001).

У цій точці виникає явище значної інертності моделі: коли здобувач усуває помилку та демонструє правильний код, знаменник формули Байєса стає настільки малим, що для повернення до нормативного порогу 0.85 здобувачу необхідно виконати понад 18–20 однотипних завдань поспіль, що знижує дидактичну ефективність навчального часу.

Особливість 3. Циклічність однотипних помилок («The Frustration Loop»). Класичний ВКТ оцінює кожну навичку ізольовано від загальної структури знань. Якщо здобувач відчуває труднощі на складній темі (наприклад, «Рекурсивний обхід дерева»), система генерує нові завдання саме за цим напрямом. Проте першопричина часто полягає у недостатній сформованості базових пререквізитів (робота зі стеком викликів або покажчиками). За відсутності механізму відступу назад за графом передумов формується циклічне повторення однотипних невдач.

Виявлені особливості обґрунтовують необхідність удосконалення класичного ВКТ шляхом динамічної адаптації його параметрів до часових характеристик написання коду, темпу правок та структури графа навчальних пререквізитів.

1.4. Децентралізовані стандарти непередаваних цифрових активів (Soulbound Tokens) та постановка завдань розробки

1.4. Децентралізовані стандарти непередаваних цифрових активів (Soulbound Tokens) та постановка завдань розробки

Після успішного опанування навчальної програми постає науково-інженерне завдання об'єктивної фіксації освітніх результатів, що забезпечує високий рівень довіри академічної спільноти та роботодавців.

Традиційні цифрові сертифікати у форматі PDF-документів із графічними елементами або QR-кодами мають вразливість до локального редагування. Централізовані реєстри на базі баз даних навчальних закладів вимагають постійної підтримки та залежать від доступності серверної інфраструктури емітента.

Застосування смарт-контрактів дозволяє перенести верифікацію досягнень у відкритий розподілений реєстр. Проте стандартні незамінні токени (NFT, ERC-721) орієнтовані на комерційний обіг і допускають вільну передачу іншій особі. Для академічної сертифікації це неприйнятно, оскільки освітні кваліфікації мають залишатися суворо персоналізованими.

Рішенням цієї проблеми є стандарт Soulbound Tokens (SBT), формалізований як ERC-5192 (Minimal Non-transferable NFTs). У смарт-контракті цього типу операція передачі активу заблокована на рівні програмного коду:

$$transferFrom(from, to, tokenId) \rightarrow revert("Err: Token is Soulbound and non\text{-}transferable") \quad (1.10)$$

Токен нерозривно зв'язується з публічною адресою здобувача освіти. Його емісія здійснюється уповноваженим вузлом освітньої платформи з можливістю відкликання у разі встановлення фактів академічної недоброчесності.

Водночас повне перенесення математичних обчислень ВКТ та збереження кожного телеметричного кванту безпосередньо в блокчейн є економічно та

технологічно недоцільним через високі транзакційні витрати (Gas Fees) та затримки підтвердження блоків.

Тому в роботі обґрунтовано гібридну дворівневу архітектуру:

- Високопродуктивний Off-chain шар: аналітичне ядро на NestJS та вебсокетах, яке в режимі реального часу (затримка < 5 мс) збирає поведінкову телеметрію, обчислює динамічні коефіцієнти BKT та проводить первинний скринінг аномалій;
- Незмінний On-chain шар: смарт-контракт у мережі другого рівня Polygon PoS, де карбується персональний непередаваний сертифікат із криптографічним аудиторським хешем сесії H_audit.

Для реалізації запропонованого підходу сформульовано такі науково-інженерні завдання:

1. Побудувати модифіковану математичну модель BKT із динамічними регуляторами вгадування P(G_t) і промаху P(S_t) та правилом обмеження числових границь знань (Clamping Rule);
2. Розробити алгоритмічні засоби перехоплення високочастотної сенсорної телеметрії дій здобувача в Monaco Editor та синтезувати інтегральний індекс академічної довіри C_trust;
3. Спроєктувати алгоритми тактичного відступу по графу навчальних пререквізитів (DAG Backtracking) та багаторівневу систему дидактичного супроводу (Remediation Scaffolding);
4. Розробити архітектуру та програмну реалізацію екосистеми з п'яти взаємопов'язаних застосунків монорепозиторію (apps/study, apps/student, apps/teacher, apps/admin, apps/api);
5. Реалізувати та верифікувати смарт-контракт SoulboundCertificate.sol стандарту ERC-5192 у блокчейн-мережі Polygon;
6. Провести експериментальне дослідження точності прогнозування результатів, стійкості до несамостійного копіювання коду та навантажувальних характеристик системи на вибірці з 1200 сесій.

РОЗДІЛ 2
МАТЕМАТИЧНЕ МОДЕЛЮВАННЯ ТА АЛГОРИТМІЗАЦІЯ АДАПТИВНОГО НАВЧАННЯ

2.1. Модифікована модель BKT із динамічними регуляторами ймовірностей P(G_t) та P(S_t)

2.1. Модифікована модель BKT із динамічними регуляторами ймовірностей P(G_t) та P(S_t)

Щоб подолати збої класичного BKT, ми відмовилися від припущення, що ймовірність вгадування P(G) та ймовірність промаху P(S) є постійними числами. У живому навчальному процесі вони змінюються щохвилини: учень може писати код зосереджено, припускаючись дрібних синтаксичних помилок, або навпаки — миттєво вставити чужий готовий блок без найменшого розуміння логіки.

Тому ми перетворили константи на динамічні регулятори P(G_t) та P(S_t), значення яких обчислюються прямо під час сесії на основі даних клієнтського сенсора.

Нехай латентний стан знання студента щодо конкретної навички на кроці t-1 описується апостеріорною ймовірністю $P(L_{t-1}) \in [0; 1]$.

Коли учень завершує крок і запускає автоматичне модульне тестування коду, бекенд отримує бінарний результат тестів $Y_t \in 0, 1$ та паралельно зчитує згенерований сенсором темпоральний вектор поведінки E_t.

Модуль BehavioralAdjuster розраховує адаптовані значення регуляторів, після чого формула Байєса оновлює рівень знання.

Якщо всі модульні тести пройдено успішно ($Y_t = 1$):

$$P(L_t \mid Y_t = 1) = (P(L_{t-1}) \cdot (1 - P(S_t)) / P(L_{t-1}) \cdot (1 - P(S_t)) + (1 - P(L_{t-1})) \cdot P(G_t)) \quad (2.1)$$

Якщо компілятор видав помилку або впав хоча б один тест ($Y_t = 0$):

$$P(L_t \mid Y_t = 0) = (P(L_{t-1}) \cdot P(S_t) / P(L_{t-1}) \cdot P(S_t) + (1 - P(L_{t-1})) \cdot (1 - P(G_t)))$$ (2.2)

Після врахування спостереження додається складова навчання в результаті спроби:

$$P(L_{t+1}^{raw}) = P(L_t \mid Y_t) + (1 - P(L_t \mid Y_t)) \cdot P(T)$$ (2.3)

де параметр P(T) задає апріорну швидкість засвоєння матеріалу (у наших курсах ми калібруємо його в діапазоні 0.15–0.20 відповідно до когнітивної складності завдання).

Як працює цей математичний механізм на практиці?

Звернемо увагу на знаменник у формулі (2.1): якщо студент скопіював код із чат-бота, алгоритм телеметрії миттєво задирає регулятор вгадування до $P(G_t) = 0.90$. Знаменник різко збільшується, і весь дріб наближається до вихідного значення $P(L_{t-1})$. Тобто навіть якщо всі юніт-тести ідеально зелені, приріст рейтингу студента виявляється практично нульовим. Система просто не нараховує бали за списаний код.

Зробимо строге математичне доведення того, що динамічний регулятор $P(G_t)$ дійсно гарантує захист від штучної накрутки.

Знайдемо частинну похідну апостеріорної ймовірності $P(L_t \mid Y_t=1)$ за змінною регулятора вгадування $P(G_t)$:

$$(\partial P(L_t \mid Y_t=1) / \partial P(G_t)) = -(P(L_{t-1}) (1 - P(L_{t-1})) (1 - P(S_t)) / [P(L_{t-1})(1 - P(S_t)) + (1 - P(L_{t-1})) P(G_t)]^2)$$ (2.4)

Проаналізуємо знаки компонентів формули (2.4):

- Початкова ймовірність знання $P(L_{t-1})$ лежить у відкритому інтервалі (0; 1), отже, добуток $P(L_{t-1}) (1 - P(L_{t-1})) > 0$;
- Ймовірність промаху строго менша одиниці ($P(S_t) < 1$), тому $(1 - P(S_t)) > 0$;
- Знаменник є квадратом дійсної суми, тобто завжди строго більший за нуль.

Оскільки перед дробом стоїть знак мінус, уся частинна похідна є строго від'ємною:

$$(\partial P(L_t \mid Y_t=1) / \partial P(G_t)) < 0, \quad \forall\ P(L_{t-1}) \in (0; 1) \qquad (2.5)$$

Аналогічно дослідимо частинну похідну за регулятором промаху P(S_t) у разі помилки здобувача (Y_t = 0):

$$(\partial P(L_t \mid Y_t=0) / \partial P(S_t)) = (P(L_{t-1}) (1 - P(L_{t-1})) (1 - P(G_t)) / [P(L_{t-1}) P(S_t) + (1 - P(L_{t-1})) (1 - P(G_t))]^2) > 0 \qquad (2.6)$$

Додатний знак похідної (2.6) наочно демонструє зворотну властивість регулятора: чим більше система впевнена, що помилка була випадковою механічною опискою сумлінного учня (зростання P(S_t)), тим менше просідає його рейтинг знання. Оцінка не обвалюється через забуту крапку з комою або друкарську помилку в назві змінної.

2.2. Математичне правило числових границь з гарантованою асимптотою (Clamping Rule)

2.2. Математичне правило числових границь з гарантованою асимптотою (Clamping Rule)

Під час практичної апробації прототипу серверної частини було виявлено специфіку поведінки формули Байєса при тривалих серіях помилок: значення P(L) стрімко наближається до нуля, внаслідок чого змінні типу Float64 у середовищі виконання V8/Node.js переходять в експоненційну форму представлення (< 10^-16). Якщо після цього здобувач опановує навчальний матеріал і демонструє правильний розв'язок, виникає ризик обнулення знаменника через втрату точності розрядів або надмірну математичну інертність оцінки.

Для усунення цієї обчислювальної нестабільності та забезпечення педагогічно обґрунтованої динаміки навчального прогресу в модель введено двостороннє операторне обмеження значень — правило Clamping:

$$P(L_{t+1}) = \max(P_{min}, \min(P_{max}, P(L_{t+1^{raw}}))) \qquad (2.7)$$

де зафіксовано граничні константи:

- P_min = 0.05 — нижня гарантована порогова асимптота оцінки знань;
- P_max = 0.999 — верхній бар'єр, що запобігає абсолютному детермінізму (1.0).

Обґрунтування межі P_min = 0.05:

Це рішення реалізує гуманістичний принцип оцінювання: навіть за наявності серії невдалих спроб система не прирівнює рівень знань студента до абсолютного нуля, припускаючи наявність мінімального потенціалу засвоєння концепту (5%), тимчасово заблокованого когнітивною втомою чи стресом.

Проілюструємо динаміку реабілітації оцінки знань за правилом (2.7). Припустимо, що показник P(L) досяг нижнього порогу P_min = 0.05. Здобувач скористався дидактичною підказкою та виконав 3 послідовні правильні спроби (при базових параметрах P(T) = 0.20, P(G) = 0.15, P(S) = 0.10):

Спроба 1:

$$P(L_1 \mid Y_1=1) = (0.05 \cdot 0.90 / 0.05 \cdot 0.90 + 0.95 \cdot 0.15) = (0.045 / 0.045 + 0.1425) \approx 0.240 \quad (2.8)$$

$$P(L_2) = 0.240 + (1 - 0.240) \cdot 0.20 = 0.392 \quad (2.9)$$

Спроба 2:

$$P(L_2 \mid Y_2=1) = (0.392 \cdot 0.90 / 0.392 \cdot 0.90 + 0.608 \cdot 0.15) = (0.3528 / 0.3528 + 0.0912) \approx 0.795 \quad (2.10)$$

$$P(L_3) = 0.795 + (1 - 0.795) \cdot 0.20 = 0.836 \quad (2.11)$$

Спроба 3:

$$P(L_3 \mid Y_3=1) = (0.836 \cdot 0.90 / 0.836 \cdot 0.90 + 0.164 \cdot 0.15) = (0.7524 / 0.7524 + 0.0246) \approx 0.968 \quad (2.12)$$

$$P(L_4) = \min(0.999, 0.968 + (1 - 0.968) \cdot 0.20) = 0.974 \quad (2.13)$$

Результат очевидний: усього за три успішні дії учень повертає впевнений статус майстерності. Модель заохочує прогрес, а не консервує старі помилки.

Математичне ядро ми винесли в чистий автономний TypeScript-клас BktEngine. Він не має зовнішніх залежностей і виконує розрахунки з константною швидкістю O(1).

Лістинг 2.1 — Реалізація ядра BktEngine з підтримкою правила Clamping

```
// apps/api/src/domain/bkt/bkt.engine.ts
export class BktEngine {
  private static readonly P_MIN = 0.05;  // Нижня асимптота знань
  private static readonly P_MAX = 0.999; // Захист від сингулярності

public static calculateNextState(
    prevL: number,
    isCorrect: boolean,
    pSlip: number,
    pGuess: number,
    pTransit: number
  ): number {
    let pObserved: number;
    if (isCorrect) {
      const num = prevL * (1 - pSlip);
      const den = num + (1 - prevL) * pGuess;
      pObserved = den === 0 ? prevL : num / den;
    } else {
      const num = prevL * pSlip;
      const den = num + (1 - prevL) * (1 - pGuess);
      pObserved = den === 0 ? prevL : num / den;
    }
    const nextL = pObserved + (1 - pObserved) * pTransit;
    return Math.min(Math.max(nextL, this.P_MIN), this.P_MAX);
  }
}
```

Завдяки простоті формули бенчмарки на серверному процесорі показують продуктивність понад мільйон операцій на секунду на одне ядро.

2.3. Алгоритми багатовимірної сенсорної телеметрії кодописання та інтегральний коефіцієнт довіри C_trust

2.3. Алгоритми багатовимірної сенсорної телеметрії кодописання та інтегральний коефіцієнт довіри C_trust

Математичний двигун BKT не може висіти в повітрі: йому потрібні об'єктивні цифри про те, як саме студент створював розв'язок. Для цього ми розробили клієнтський сенсор, вбудований у редактор коду Monaco.

Коли студент відкриває задачу, сенсор починає формувати темпоральний вектор сесії:

$$E_t = \langle \Delta t_solve, WPM, N_backspaces, N_paste, R_paste, T_first_key, R_blur \rangle$$

(2.14)

Розберемо фізичний зміст кожної метрики:

- Δt_solve — загальний час перебування над задачею (від першого рендеру до кліку на кнопку перевірки);
- WPM (Words Per Minute) — швидкість набору тексту;
- N_backspaces — кількість натискань клавіш Backspace/Delete. Це критично важливий показник: живий програміст постійно виправляє описки, думає, повертається назад;
- N_paste — скільки разів було зафіксовано подію вставки з буфера;
- R_paste = L_pasted / L_total — відношення обсягу вставлених символів до загального фінального обсягу файлу;
- T_first_key — пауза між появою тексту умови та першим натисканням клавіші (латентність читання й осмислення);
- R_blur — частка часу, коли вкладка браузера була неактивною (студент перемикався на сторонні вікна чи месенджери).

1. Розрахунок регулятора вгадування P(G_t).

Якщо учень працює самостійно (R_paste < 0.20, час розв'язання перевищує 30 секунд), ми тримаємо регулятор на базовому мінімумі P(G_t) = 0.15.

Але якщо сенсор фіксує масивну вставку коду, ми вмикаємо штрафну логістичну криву:

$$P(G_t) = P(G_base) + (P(G_max) - P(G_base) / 1 + \exp(-12 \cdot (R_paste - 0.50)))$$

(2.15)

де P(G_base) = 0.15, а стеля штрафу P(G_max) = 0.92.

Крутизна коефіцієнта 12 забезпечує різкий перегин: щойно частка вставки перетинає 50–60%, параметр P(G_t) стрибає до 0.90, що автоматично позбавляє студента балів за списаний код.

2. Розрахунок регулятора промаху P(S_t).

З іншого боку, ми повинні підтримати сумлінного студента, який довго й наполегливо шукав помилку. Якщо учень провів над задачею понад дві хвилини і має десятки правок клавішею Backspace, карати його за випадкову кому в коді негуманно.

Ми підвищуємо ймовірність промаху:

$$P(S_t) = P(S_base) + \beta_s \cdot \lesssim ft(1 - exp\lesssim ft(-(\Delta\ t_solve / \tau_norm))) \cdot (N_backspaces / N_keystrokes + 1) \quad (2.16)$$

де P(S_base) = 0.10, ваговий коефіцієнт β_s = 0.30, а характерний час нормалізації τ_norm = 180 секунд. Якщо студент активно правив свій код, регулятор зростає до 0.35–0.40, що захищає рейтинг від обвалу при помилці в тестах.

3. Підсумковий індекс академічної довіри C_trust.

Для формування загального вердикту про доброчесність ми звели показники в інтегральний індекс:

$$C_trust = w_1 \cdot \psi(WPM) + w_2 \cdot (1 - R_paste) + w_3 \cdot \varphi(T_first_key) + w_4 \cdot (1 - R_blur) \quad (2.17)$$

Емпірично підібрані вагові коефіцієнти становлять: w_1 = 0.25, w_2 = 0.35, w_3 = 0.20, w_4 = 0.20.

Функція правдоподібності швидкості набору ψ(WPM) моделюється гаусовим куполом навколо середньої швидкості друку коду:

$$\psi(WPM) = exp\lesssim ft(-((WPM - 65)^2 / 2 \cdot 35^2)) \quad (2.18)$$

Якщо WPM перевищує 200 (що неможливо для осмисленого набору нового тексту) або навпаки падає нижче 5 слів за хвилину, функція прямує до нуля.

А функція латентності першого натискання φ(T_first_key) захищає від миттєвого сабміту:

$$\varphi(T_first_key) = (1 / 1 + exp(-0.1 \cdot (T_first_key - 5))) \quad (2.19)$$

Якщо учень відправив код через 2 секунди після відкриття задачі, значення функції становить менше 0.40, що миттєво роняє весь індекс C_trust нижче порогу доброчесності 0.75.

2.4. Алгоритм тактичного відступу за графом пререквізитів (DAG Backtracking) та трирівневе дидактичне риштування

2.4. Алгоритм тактичного відступу за графом пререквізитів (DAG Backtracking) та трирівневе дидактичне риштування

Коли студент не може впоратися з задачею, найгірше, що може зробити навчальна система — це знову і знову видавати йому той самий тип завдань. Щоб розірвати петлю фрустрації (The Frustration Loop), ми розробили алгоритм тактичного відступу по графу знань курсу.

Предметна область описується орієнтованим ациклічним графом G = (V, E), де вершини V — це конкретні навички, а ребра E задають строгі пререквізити (тему u потрібно знати, щоб зрозуміти тему v).

1. Як працює DAG Backtracking?

Якщо здобувач припускається трьох послідовних помилок підряд ($k \geq 3$) на поточній вершині v_curr, а його оцінка знань опустилася нижче критичного порогу 0.25, алгоритм зупиняє практичну сесію.

Система виконує такі кроки:

Крок 1. Знаходить усіх безпосередніх предків поточної теми в графі:

$$Pred(v_curr) = \{u \in V \mid (u, v_curr) \in E\} \tag{2.20}$$

Крок 2. Серед знайдених пререквізитів обирає вузол із найгіршим рівнем засвоєння:

$$u_remedy = \operatorname{argmin}_{u \in Pred(v_curr)} P(L_u) \tag{2.21}$$

Крок 3. Замість чергової складної задачі система м'яко перемикає студента в дидактичний застосунок apps/study на спеціальний короткий теоретичний модуль за темою u_remedy.

2. Трирівнева модель дидактичного риштування (Remediation Scaffolding). Ми принципово відмовилися від кнопки «Показати розв'язок». Замість цього система допомагає поступово:

- Рівень 1 (Сократівська підказка, Socratic Hint): навідне запитання щодо логіки алгоритму (наприклад: «Що станеться, якщо масив на вході буде порожнім?»);
- Рівень 2 (Картка ментальної омани, Misconception Card): розбір типової когнітивної пастки з контрастним порівнянням правильного і помилкового підходів;
- Рівень 3 (Когнітивне розвантаження, Cognitive Cool-Down): якщо аналізатор фіксує ознаки когнітивної перевтоми здобувача (індекс втоми $I_fatigue > 0.70$ на основі аномального зростання міжсимвольних пауз), інтерфейс тимчасово призупиняє доступ до поточного завдання на 15 хвилин, рекомендуючи інтервал відпочинку для відновлення зосередженості.

Правила переходів представлено у матриці рішень (таблиця 2.1).

Таблиця 2.1 — Матриця керуючих рішень адаптивного рушія ремедіації

Поточний стан P(L)	**Показник довіри C_trust**	**Кількість помилок k**	**Керуюче педагогічне рішення системи**
$P(L) \geq 0.85$	$C_trust \geq 0.75$	0	Фіксація майстерності, відкриття наступних вершин графа DAG
$P(L) \geq 0.85$	$C_trust < 0.50$	0	Блокування переходу, перенаправлення в чергу ReviewQueueInspector
$0.40 \leq P(L) < 0.85$	Будь-який	1 - 2	Активація адаптивної підказки Рівня 1 (Socratic Hint)
$P(L) < 0.25$	Будь-який	≥ 3	Активація DAG Backtracking, перехід на повторення пререквізиту
Будь-який	$I_fatigue > 0.70$	≥ 4	Блокування на перерву 15 хв (Cognitive Cool-Down).

2.5. Математична та криптографічна модель аудиту навчальних сесій і верифікації сертифікатів

2.5. Математична та криптографічна модель аудиту навчальних сесій і верифікації сертифікатів

Останній крок навчального циклу — формування незаперечного доказу того, що студент дійсно засвоїв матеріал.

Смарт-контракт емісії сертифіката активується за одночасного виконання трьох суворих критеріїв:

1. Критерій майстерності: для всіх навичок навчального модуля досягнуто поріг знань 0.85:

$$\forall\ v \in V_module : P(L_v) \geq \theta_mastery,\ quad\ \theta_mastery = 0.85 \qquad (2.22)$$

2. Критерій доброчесності: середній індекс довіри сесій модуля перевищує мінімальний поріг:

$$barC_trust = (1 / M)\ \sum_i=1\^M\ C_trust\^(i) \geq 0.75 \qquad (2.23)$$

3. Критерій ручної перевірки: в черзі ReviewQueueInspector відсутні блокуючі відхилення викладачем.

Коли всі три умови виконано, оракул формує детермінований аудиторський хеш H_audit за алгоритмом Keccak-256:

$$H_audit = keccak256(abi.encodePacked(addr_student, id_module, barP_L, barC_trust, timestamp, nonce)) \qquad (2.24)$$

Цей хеш назавжди вкарбовується у блокчейн Polygon PoS через виклик методу `mintCertificate`. Зв'язка публічної адреси студента, ідентифікатора курсу, часової мітки та середнього індексу довіри гарантує: ні викладач, ні системний адміністратор не зможуть «домалювати» сертифікат людині, яка не пройшла практичний курс у системі.

РОЗДІЛ 3
АРХІТЕКТУРА ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІНТЕЛЕКТУАЛЬНОЇ EDTECH-СИСТЕМИ

3.1. Загальна мікросервісна та мікрофронтендна архітектура монорепозиторію

3.1. Загальна мікросервісна та мікрофронтендна архітектура монорепозиторію

Програмний комплекс Smart-BKT-Chain спроєктовано за сучасними принципами компонентної мікрофронтендної та гексагональної мікросервісної архітектури. Весь програмний стек організовано в єдиний керований монорепозиторій під управлінням диспетчера пакетів npm workspaces та конвеєрного оркестратора збірки Turborepo.

Такий підхід забезпечує критично важливі інженерні переваги для високонавантаженої освітньої платформи:

1. Сувору наскрізну статичну типізацію між клієнтськими та серверними підсистемами завдяки спільному доменному пакету `@sbc/types`. Будь-яка зміна структури телеметричного кванту чи стану моделі BKT негайно верифікується компілятором TypeScript на обох кінцях з'єднання;
2. Відсутність копіювання коду та уніфікацію поведінки користувацького інтерфейсу через винесення стандартизованих компонентів у спільну бібліотеку `@sbc/ui` (зокрема базового структурного блоку `DrawerSection` за стандартом безрамкових бічних панелей);
3. Централізоване декларативне керування реляційною схемою бази даних через пакет `@sbc/database` на основі Prisma ORM із підтримкою строго типізованих міграцій;
4. Повну ізоляцію середовищ виконання: кожен веб-застосунок монорепозиторію має власний незалежний життєвий цикл збірки, тестування та запуску, що усуває взаємний вплив при локальній розробці та горизонтальному масштабуванні.

Загальне архітектурне дерево компонентів і пакетів монорепозиторію наведено в лістингу 3.1.

Лістинг 3.1 — Архітектурна структура монорепозиторію платформи Smart-BKT-Chain

```
smart-bkt-chain/
├── apps/
│   ├── study/         # Порт 3006: Дидактичний простір (Zen Focus, Misconceptions,
Playground)
│   ├── student/       # Порт 3002: Тренажер учня (Monaco Editor, Skill DAG Map,
Live Telemetry)
│   ├── teacher/       # Порт 3003: Моніторинг викладача (BKT Heatmap, Review Queue
Inspector)
│   ├── admin/         # Порт 3004: CMS Методиста (Skill DAG Visual Builder, Course
Manager)
│   └── api/               # Порт 3000: Гексагональний NestJS Core (WebSockets,
BktEngine, BullMQ)
├── packages/
│   ├── database/     # Prisma ORM, PostgreSQL schema, міграції
│   ├── types/        # Спільні TypeScript DTO контракти (@sbc/types)
│   └── ui/           # Спільна бібліотека компонентів (@sbc/ui, DrawerSection)
└── contracts/        # Solidity смарт-контракти (SoulboundCertificate.sol)
```

Функціональний та технологічний розподіл обов'язків веб-застосунків екосистеми зведено в табл. 3.1.

Таблиця 3.1 — Функціональний та технологічний розподіл застосунків комплексу

Застосунок	**Порт / URL**	**Роль у навчальному процесі**	**Ключові технологічні модулі та бібліотеки**
apps/study	localhost:3006	Здобувачі (дидактика)	Next.js 16, Zen Focus Reading Engine, CodePlayground, Web Worker
apps/student	localhost:3002	Здобувачі (практика)	Monaco Editor, useBehavioralTelemetry, SkillDagMap, Socket.IO
apps/teacher	localhost:3003	Викладачі та тьютори	BKT Heatmap, ReviewQueueInspector, AnomalyRadar, LLM Review
apps/admin	localhost:3004	Методисти курсів	Skill DAG Visual Builder, Course Manager, BKT Configurator

Застосунок	Порт / URL	Роль у навчальному процесі	Ключові технологічні модулі та бібліотеки
apps/api	localhost:3000	Центральний бекенд	NestJS 10, WebSocket Gateway, BktEngine, BullMQ, Prisma ORM.

Міжпакетна взаємодія базується на суворому дотриманні політики інверсії залежностей: застосунки верхнього рівня (apps/*) не можуть імпортувати внутрішній код інших застосунків напряму. Будь-який спільний контракт, DTO або інтерфейс попередньо реєструється в доменному пакеті @sbc/types.

Для стандартизації бічних панелей (Side Drawers) в усіх трьох кабінетах (student, teacher, admin) розроблено єдиний дизайн-стандарт «Панель як контейнер, вміст як секції». Панель слугує чистим структурним контейнером, а весь функціональний вміст групується у згортанні акордеонні секції через компонент DrawerSection із мінімалістичними індикаторами розгортання Plus та Minus.

3.2. Гексагональне аналітичне ядро бекенду (apps/api), WebSocket-шлюзи та схема бази даних

3.2. Гексагональне аналітичне ядро бекенду (apps/api), WebSocket-шлюзи та схема бази даних

Серверна частина системи apps/api реалізована за архітектурним патерном «Порти та Адаптери» (Hexagonal Architecture). Доменна логіка оцінювання знань BKT, розрахунку телеметричних штрафів та обходу графа повністю ізольована від зовнішніх транспортних фреймворків (HTTP-контролерів, WebSocket-гейтвеїв) та деталей збереження в реляційній базі даних.

1. Транспортний шар потокової телеметрії (WebSocket Gateway).

Для мінімізації мережевих затримок клієнтський тренажер комунікує з бекендом через бідирекційний бінарний сокет-канал Socket.IO.

Кожні 1500 мс клієнт надсилає телеметричний фрейм TELEMETRY_TICK, що містить поточні показники буфера, міжсимвольні затримки та зсув курсору. Обробник гейтвею виконує валідацію схеми через class-validator за менш ніж 0.4 мс.

2. Контракти передачі даних пакета @sbc/types.

Для усунення розриву типізації між TypeScript-клієнтами та NestJS-ендпоінтами в пакеті @sbc/types зафіксовано суворі структури даних: DTO конфігурації BKT, DTO стану навички та DTO телеметричних квантів подій.

Лістинг 3.2 — Спільні DTO-контракти типізації BKT та телеметрії (@sbc/types)

```
// packages/types/src/bkt.types.ts
export interface BktParametersDto {
  pInit: number;    // P(L_0) - початкова ймовірність знання
  pTransit: number; // P(T) - ймовірність переходу у стан знання
  pSlip: number;    // P(S) - ймовірність помилки при знанні
  pGuess: number;   // P(G) - ймовірність вгадування без знання
}

export interface BktStateDto {
  studentId: string;
  skillId: string;
  pKnown: number;          // P(L_n) - актуальна ймовірність [0..1]
  isMastered: boolean;     // P(L_n) >= 0.85
  totalAttempts: number;
  correctAttempts: number;
}

export interface BktUpdateResultDto {
  previousPknown: number;
  newPknown: number;
  isMastered: boolean;
  delta: number;
  skillId: string;
  adjustedParams?: {
    usedSlip: number;
    usedGuess: number;
    anomalyDetected: boolean;
  };
}
```

3. Захист від спаму спроб та забезпечення ідемпотентності (Idempotency Guard).

Під час практичного використання було виявлено загрозу багаторазового надсилання одного й того самого розв'язку з метою штучної накрутки значень P(L).

Для запобігання цій аномалії розроблено алгоритм ідемпотентності:

1) Бекенд обчислює SHA-256 хеш отриманого програмного коду:

$$H_code = sha256(code \parallel taskId \parallel studentId) \quad (3.1)$$

2) Якщо надходить запит із ідентичним значенням H_code протягом захисного вікна (30 секунд), або якщо код повністю збігається з попереднім успішним кроком, сервер повертає статус DUPLICATE_SUBMISSION_IGNORED без виклику формули оновлення BKT.

4. Структура реляційної моделі даних (Prisma Schema).

Для персистентного збереження графа знань, сесій, телеметрії та черги перевірки спроєктовано оптимізовану реляційну схему в базі даних PostgreSQL.

Лістинг 3.3 — Фрагмент декларації Prisma-схеми даних телеметрії та BKT-станів

```
// packages/database/prisma/schema.prisma
model StudentBktState {
  id              String   @id @default(uuid())
  studentId       String
  skillId         String
  pKnown          Float    @default(0.15)
  consecutiveFails Int     @default(0)
  lastUpdated     DateTime @updatedAt

@@unique([studentId, skillId])
  @@index([studentId])
}

model TelemetrySession {
  id              String   @id @default(uuid())
  studentId       String
  taskId          String
  timeToSolveMs   Int
  wpm             Float
  pasteRatio      Float
  backspaceCount  Int
  trustScore      Float
  isClean         Boolean  @default(true)
  auditHash       String?
  createdAt       DateTime @default(now())

@@index([studentId, taskId])
}

model ReviewSubmission {
  id              String   @id @default(uuid())
  studentId       String
  taskId          String
  code            String   @db.Text
  version         Int      @default(1)
  status          String   @default("PENDING_REVIEW")
  reviewerNotes   String?  @db.Text
  createdAt       DateTime @default(now())
}
```

Всі операції над станами обгорнуті в транзакційні блоки Prisma Interactive Transactions, що гарантує відсутність станів гонитви (Race Conditions) при паралельних сабмітах.

3.3. Клієнтські підсистеми практичного кодування та методичного керування (apps/student, apps/teacher, apps/admin)

3.3. Клієнтські підсистеми практичного кодування та методичного керування (apps/student, apps/teacher, apps/admin)

Клієнтські застосунки побудовані на основі компонентної архітектури React та Next.js 16.

1. Тренажер програмування apps/student (порт 3002).

Центральним робочим простором учня є редактор коду Monaco Editor. Для збору інформації без зниження продуктивності інтерфейсу (60 FPS) розроблено кастомний React-хук useBehavioralTelemetry. Хук перехоплює події:

- onDidChangeModelContent: реєстрація операцій вставки тексту та очищення буфера обміну;
- onKeyDown: вимірювання міжсимвольних латентностей (Inter-keystroke intervals);
- onDidBlurEditorWidget: фіксування втрати фокусу вікна браузера.

Візуалізація графа знань реалізована за допомогою компонента SkillDagMap, де кольорове кодування вершин динамічно відображає ймовірність засвоєння P(L) від червоного ($P(L) < 0.30$) через жовтий до смарагдово-зеленого ($P(L) \geq 0.85$).

2. Робоче місце викладача apps/teacher (порт 3003).

Містить когортну теплову карту успішності (BKT Heatmap), радар виявлення аномалій академічної доброчесності та інспектор черги перевірки робіт ReviewQueueInspector.

Для запобігання засміченню черги перевірки реалізовано скінченний автомат статусів спроби:

- PENDING_REVIEW — очікує перевірки викладачем або LLM-суддею;

- SUPERSEDED — автоматично замінено новою версією розв'язку того самого учня;
- APPROVED — розв'язок схвалено із зарахуванням балів;
- REJECTED — роботу повернуто на доопрацювання з коментарем.

3. Компонент інспекції черги ReviewQueueInspector.

Компонент організовано згідно зі стандартом бічних панелей DrawerSection. Викладач бачить не просто текст коду, а синхронізовану теплову карту активності: підозрілі вставки позначаються червоним підсвічуванням, графік WPM показує неприродні сплески швидкості, а зведена оцінка LLM-судді пропонує попередній вердикт із коментарем.

4. CMS методиста курсу apps/admin (порт 3004).

Забезпечує інтерактивне конструювання графа пререквізитів курсу (Skill DAG Visual Builder), налаштування параметрів BKT (P(L_0), P(T), P(G), P(S)) та додавання навчальних модулів. Методист може зв'язувати вершини пререквізитів простим перетягуванням (Drag-and-Drop) та візуально верифікувати ациклічність графа.

3.4. Архітектура навчального простору первинної дидактики та рушія фокусування (apps/study та Zen Focus Reading Engine)

3.4. Архітектура навчального простору первинної дидактики та рушія фокусування (apps/study та Zen Focus Reading Engine)

На відміну від систем, орієнтованих лише на контроль, платформа містить повнофункціональний простір теоретичного навчання та первинної дидактики — застосунок apps/study (порт 3006).

1. Рушій комфортного читання (Zen Focus Reading Engine).

Для зменшення когнітивного навантаження здобувача розроблено інтерактивний рушій Zen Focus Reading Engine:

- Динамічне підсвічування активного смислового абзацу або рядка коду (Line Highlighting) із м'яким затемненням периферійного тексту;
- Можливість персонального вибору кегля (14–22 px), міжрядкового інтервалу

(1.4–1.8) та колірних тем (глибока темна, нічна сапфірова, класична світла) з гарантованою контрастністю WCAG AAA;

- Бездистракційний повноекранний режим читання (Distraction-Free Mode).

2. Інтерактивна пісочниця CodePlayground.

У кожну теоретичну лекцію інтегровано живі пісочниці, де учень може експериментувати з кодом мовами TypeScript/JavaScript/Python безпосередньо у вікні браузера без необхідності налаштування локального середовища розробки. Виконання коду відбувається в безпечному Web Worker, ізольованому від основного DOM-дерева сторінки.

3. Картки типових ментальних оман (Misconception Cards).

Усі картки спроєктовано згідно зі стандартом акордеонних секцій @sbc/ui DrawerSection. Вони наочно протиставляють хибний патерн розуміння правильному алгоритмічному рішенню та містять контрольне питання на швидке закріплення. Студент бачить помилкове міркування (наприклад, «метод array.sort() сортує числа за числовим значенням»), пояснення проблеми (лексикографічне сортування за замовчуванням) та вірний приклад із компаратором.

3.5. Смарт-контракт децентралізованої сертифікації SoulboundCertificate у мережі Polygon

3.5. Смарт-контракт децентралізованої сертифікації SoulboundCertificate у мережі Polygon

Для криптографічної фіксації досягнень здобувачів освіти розроблено комплект смарт-контрактів мовою Solidity (версія 0.8.20) для EVM-сумісної мережі Polygon PoS.

Смарт-контракт SoulboundCertificate.sol успадковує стандарти ERC-721 та інтерфейс IERC5192 (Minimal Non-transferable Soulbound NFTs).

Лістинг 3.4 — Смарт-контракт непередаваного сертифіката SoulboundCertificate.sol

```
// contracts/SoulboundCertificate.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract SoulboundCertificate is ERC721, Ownable {
    event Locked(uint256 tokenId);

mapping(uint256 => bytes32) public auditHashes;
    mapping(uint256 => bool) public isLocked;

constructor() ERC721("SmartBKT Soulbound Credential", "SBC") Ownable(msg.sender) {}

function mintCertificate(address to, uint256 tokenId, bytes32 auditHash) external
onlyOwner {
        _safeMint(to, tokenId);
        auditHashes[tokenId] = auditHash;
        isLocked[tokenId] = true;
        emit Locked(tokenId);
    }

function _beforeTokenTransfer(address from, address to, uint256, uint256) internal
pure override {
                require(from == address(0) || to == address(0), "Err: SBT is
non-transferable");
    }
}
```

Перевизначення хука _beforeTokenTransfer повністю унеможливлює передачу або перепродаж сертифіката між гаманцями користувачів: токен може бути створений лише шляхом емісії від нульової адреси (address(0)) або спалений емітентом у разі виявлення порушень доброчесності.

РОЗДІЛ 4
ОРГАНІЗАЦІЯ ПРОЦЕСУ РОЗРОБКИ ТА УПРАВЛІННЯ ВИМОГАМИ ДО ПРОГРАМНОГО КОМПЛЕКСУ

4.1. Гнучка методологія розробки, архітектура компонентів та декомпозиція епіків у системі Jira

4.1. Гнучка методологія розробки, архітектура компонентів та декомпозиція епіків у системі Jira

Створення багатокомпонентної освітньої екосистеми Smart-BKT-Chain, що поєднує гексагональне аналітичне бекенд-ядро, п'ять спеціалізованих веб-застосунків монорепозиторію, шар реляційних баз даних та смарт-контракти децентралізованої сертифікації, вимагало впровадження суворої інженерної системи управління життєвим циклом розробки програмного забезпечення (Software Development Life Cycle — SDLC).

Управління розробкою проєкту організовано за адаптивною методологією Agile (Scrum/Kanban) з використанням системи відстеження завдань Atlassian Jira Cloud. Для усунення хаотичності та забезпечення наскрізної простежуваності (Traceability) від високорівневих вимог до конкретних рядків вихідного коду весь функціональний обсяг робіт монорепозиторію було структуровано за допомогою системи архітектурних компонентів (Jira Components).

Кожен компонент закріплює сферу інженерної відповідальності та має фіксований префікс, наведений у табл. 4.1.

Таблиця 4.1 — Архітектурна класифікація компонентів монорепозиторію в системі Jira

Код префікса	Компонент у системі Jira	Сфера інженерної відповідальності (Scope)	Робоча директорія монорепозиторію
INF	DevOps-Infra	Конфігурація монорепозиторію, Turborepo, Docker, CI/CD, документація	/, /.github/, /docker/, apps/docs
SH	Shared-Packages	Спільна схема БД (Prisma), доменні контракти типізації (@sbc/types, @sbc/ui)	packages/database, packages/types, packages/ui
BK	Backend-API	Ядро NestJS, автентифікація (JWT/OAuth2), ролі RBAC, REST-контролери	apps/api/src
BKT	BKT-Engine-AI	Математичний рушій BKT, корекція ймовірностей, черги BullMQ, телеметрія	apps/api/src/domain/bkt
LLM	LLM-Assessment	Автоматизована перевірка коду та відкритих відповідей, LLM-суддя	apps/api/src/domain/assessment
W3	Web3-Blockchain	Смарт-контракти Solidity (ERC-5192), тестове середовище Foundry	packages/contracts
ST	Student-Frontend	Кабінет Студента (Next.js 16, Monaco Editor, граф навичок, телеметрія)	apps/student
TC	Teacher-Frontend	Кабінет Викладача (BKT Heatmap, черга ReviewQueueInspector, аналітика)	apps/teacher
AD	Admin-CMS	Панель Методиста (Skill DAG Visual Builder, конструктор курсів)	apps/admin
LD	Landing-Web	Публічний портал презентації платформи та Web3-авторизація	apps/web
MS	Master-Study	Дидактичний портал вивчення теорії, Zen Focus Reading Engine	apps/study.

На основі зазначеної компонентної моделі життєвий цикл розробки декомпозовано на 10 взаємопов'язаних функціональних епіків:

1. SBC-EP-INF (Monorepo & CI/CD Setup) — розгортання робочого простору npm workspaces під управлінням Turborepo, контейнеризація PostgreSQL, налаштування лінтерів ESLint/Prettier та автоматизації Playwright E2E;
2. SBC-EP-SH (Database Modeling & Shared Packages) — проєктування нормалізованої реляційної схеми Prisma ORM, бібліотеки UI-компонентів @sbc/ui та наскрізних DTO-контрактів @sbc/types;
3. SBC-EP-BK (Authentication & Core API) — розробка механізмів авторизації, перевірки сесій та захисту маршрутів NestJS Guards;
4. SBC-EP-BKT (Bayesian Knowledge Tracing & Behavioral Telemetry) — алгоритмізація розрахунку матриць BKT, динамічних коефіцієнтів P(G_t) і P(S_t) та збору телеметрії кодописання;
5. SBC-EP-LLM (Automated Code Assessment & Open-Text Evaluation) — конвеєр автоматичного тестування коду та рубрикованої оцінки текстових відповідей;
6. SBC-EP-W3 (Web3 Infrastructure & Smart Contracts) — розробка смарт-контрактів стандарту Soulbound Tokens (ERC-5192) у середовищі Foundry;
7. SBC-EP-ST (Student Portal & Skill Tree Map) — розробка клієнтського інтерфейсу учня, редактора Monaco Editor та сенсорного хука useBehavioralTelemetry;
8. SBC-EP-TC (Teacher Portal & Analytics CRM) — робоче місце педагога з когортною аналітикою BKT Heatmap та чергою ручної перевірки;
9. SBC-EP-AD (Admin Portal & Knowledge DAG Builder) — візуальний редактор орієнтованого ациклічного графа навчальних пререквізитів;
10. SBC-EP-MS (Master Study Space & Reading Engine) — дидактичний застосунок теоретичного навчання з рушієм комфортного читання Zen Focus Reading Engine.

Така структура гарантує повну відповідність розроблюваного коду завданням проєкту, унеможливлюючи появу неконтрольованого функціоналу.

4.2. Регламент контролю версій, семантичне маркування коммітів (Jira Conventional Commits) та автоматизовані Git Hooks

4.2. Регламент контролю версій, семантичне маркування коммітів (Jira Conventional Commits) та автоматизовані Git Hooks

Для підтримки високої культури розробки та виключення людського фактору в командній взаємодії розроблено та закріплено спеціальний регламент версіонування вихідного коду (Git Workflow).

1. Стратегія розгалуження (Branching Strategy).

Робота над кожною новою функціональністю або дефектом здійснюється в ізольованих гілках репозиторію, назва яких строго відповідає ідентифікатору завдання в Jira:

$$branch_name = type/SBC\text{-}ID\text{-}short\text{-}description \quad (4.1)$$

Приклади найменування гілок: `feature/SBC-66-monaco-telemetry`, `bugfix/SBC-82-task-transition`, `refactor/SBC-117-dag-editor`.

2. Стандарт оформлення повідомлень коммітів (Jira Conventional Commits).

Кожен комміт у репозиторії зобов'язаний містити зв'язку з номером завдання, кодом епіку, типом модифікації та коротким описом англійською мовою у наступному форматі:

$$commit_message = "[SBC\text{-}XXX][EP\text{-}YYY] <type>: <description>" \quad (4.2)$$

де дозволеними типами інженерних змін є:

- `feat` — додавання нової функціональності або нового інтерфейсного компонента;
- `fix` — виправлення виявленої помилки у бізнес-логіці чи обчисленнях;
- `docs` — створення або оновлення інженерної документації монорепозиторію;

- `style` — модифікація стилів оформлення, відступів та вирівнювання без зміни поведінки коду;
- `refactor` — оптимізація внутрішньої архітектури без зміни зовнішнього контракту;
- `test` — додавання модульних тестів (Jest) або наскрізних сценаріїв (Playwright E2E);
- `chore` — оновлення конфігурацій збірки, пакетів залежностей або допоміжних скриптів.

3. Автоматизований захист через локальні перехоплювачі (Git Hooks).

Щоб унеможливити потрапляння некоректних коммітів у репозиторій, розроблено багаторівневий механізм верифікації на базі хука `commit-msg` (.git/hooks/commit-msg).

При кожній спробі виконання операції `git commit` перехоплювач передає повідомлення валідаційному скрипту scripts/validate_commit_msg.js, реалізацію якого наведено в лістингу 4.1.

Лістинг 4.1 — Скрипт автоматичної валідації повідомлень коммітів (scripts/validate_commit_msg.js)

```
// scripts/validate_commit_msg.js
const fs = require('fs');

const msgFile = process.argv[2];
const commitMsg = fs.readFileSync(msgFile, 'utf8').trim();

// Регулярний вираз перевірки стандарту Jira Conventional Commits
const commitRegex = /^\[SBC-\d+\]\[EP-(INF|SH|BK|BKT|LLM|W3|ST|TC|AD|LD|MS)\]
(feat|fix|docs|style|refactor|test|chore): .+/;

if (!commitRegex.test(commitMsg)) {
    console.error('\x1b[31m%s\x1b[0m', 'ПОМИЛКА: Некоректний формат повідомлення
комміту!');
  console.error('Очікуваний формат: [SBC-XXX][EP-YYY] <type>: <description>');
   console.error('Приклад: [SBC-66][EP-ST] feat: implement monaco editor telemetry
hook');
  process.exit(1);
}

console.log('\x1b[32m%s\x1b[0m', 'Повідомлення комміту успішно валідовано.');
process.exit(0);
```

Якщо регулярний вираз не збігається з текстом повідомлення, процес комміту негайно переривається з ненульовим кодом повернення (exit code 1), а

розробник отримує детальну підказку з правилами формування заголовка. Це гарантує 100% відповідність історії змін стандартам інженерної культури.

4.3. Автоматизований експорт та синхронізація артефактів проєктної аналітики

4.3. Автоматизований експорт та синхронізація артефактів проєктної аналітики

Для забезпечення повної прозорості ходу розробки перед замовником та академічним керівництвом було розроблено алгоритм двосторонньої синхронізації між локальним сховищем завдань і зовнішньою хмарною системою Jira.

Вся історія виконаних завдань реєструється в центральному реєстрі docs/tasks/registry.md. Для завантаження завдань у хмарний сервіс Jira Cloud використовується автоматизований експортер, що формує нормалізований файл docs/tasks/jira_import.csv відповідно до офіційної специфікації Atlassian Jira CSV Importer.

Формат структури даних для імпорту в Jira наведено в табл. 4.2.

Таблиця 4.2 — Специфікація структури експорту артефактів завдань у форматі Jira CSV

Поле заголовка CSV	**Тип даних у системі Jira**	**Опис поля та призначення**	**Приклад значення в реєстрі проєкту**
Issue Key	String (ID)	Унікальний ідентифікатор завдання в проєкті	SBC-66
Epic Link	String (Ref)	Прив'язка завдання до відповідного функціонального епіку	SBC-EP-ST
Issue Type	Enum	Класифікація завдання (Story, Task, Bug, Docs, Spike)	Story

Поле заголовка CSV	Тип даних у системі Jira	Опис поля та призначення	Приклад значення в реєстрі проєкту
Summary	Text	Короткий опис суті завдання англійською мовою	implement Monaco Code Editor with telemetry
Status	Enum	Поточний стан виконання (Done, In Progress, To Do)	Done
Resolved Date	Date (ISO)	Дата фактичного виконання та перевірки завдання	2026-10-03
Commit Hash	String (SHA-1)	Хеш коміту в репозиторії Git для контролю цілісності	db51cda.

Синхронізація виконується автоматично за допомогою спеціального скрипта генерації артефактів scripts/generate_jira_artifacts.js, який перевіряє цілісність посилань на комміти та виявляє неузгодженості в нумерації завдань.

4.4. Критерії готовності функціоналу (Definition of Done) та інженерна верифікація мікро-кроків

4.4. Критерії готовності функціоналу (Definition of Done) та інженерна верифікація мікро-кроків

Управління якістю коду в проєкті Smart-BKT-Chain базується на концепції мікро-кроків (Micro-Step Development Execution) та суворих критеріях готовності (Definition of Done — DoD). Жодне інженерне завдання не може бути закрите зі статусом Done без виконання обов'язкового чек-листа верифікації.

1. Загальні критерії готовності системи (Global Definition of Done):

- Компіляція та типізація: успішне проходження перевірки типів TypeScript у всіх застосунках та пакетах (`npm run typecheck` завершується з кодом 0);
- Статичний аналіз: відсутність помилок та критичних попереджень лінтера (`npm run lint`);
- Модульне тестування: проходження юніт-тестів для доменних модулів BKT та

алгоритмів телеметрії із покриттям коду понад 85%;

- Скрізне тестування: успішне виконання автоматизованих тестів Playwright E2E (`npm run test:e2e`) у реальному headless-браузері для критичних сценаріїв взаємодії учня з тренажером;

- Оновлення документації: актуалізація денного звіту прогресу docs/daily_progress.md, детального інженерного журналу docs/detailed_implementation_log.md та перекомпіляція JSON-структур документації (`python scripts/compile_docs.py`).

2. Специфічні вимоги за підсистемами комплексу зведено в табл. 4.3.

Таблиця 4.3 — Специфічні критерії готовності функціональних підсистем комплексу

Архітектурний рівень	**Ключові перевірочні критерії (DoD)**	**Інструменти автоматизації перевірки**
Frontend (ST, TC, AD, MS)	Відсутність Hydration Mismatch, адаптивність інтерфейсу, відповідність стандарту бічних панелей DrawerSection із індикаторами Plus/Minus	Playwright, Next.js Compiler, TailwindCSS
Backend Core (BK, BKT, LLM)	Валідація вхідних даних через class-validator, захист від повторних сабмітів, транзакційність Prisma Interactive Transactions	Jest, NestJS TestingModule, Supertest
Web3 Smart Contracts (W3)	Компіляція Solidity 0.8.20+ без попереджень, 100% покриття тестів, перевірка непередаваності Soulbound токена через require	Foundry, Forge Test, Gas Reporter.

Завдяки впровадженню описаної інженерної системи розробки було забезпечено високу надійність програмного коду, усунуто можливість виникнення станів гонитви при багатокористувацькій роботі та досягнуто повної прозорості реалізації поставлених завдань магістерської роботи.

РОЗДІЛ 5
ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ ТА ОЦІНКА ЕФЕКТИВНОСТІ РОЗРОБЛЕНОЇ СИСТЕМИ

5.1. Методика організації комп'ютерного експерименту та характеристика тестового датасету

5.1. Методика організації комп'ютерного експерименту та характеристика тестового датасету

Для всебічної емпіричної валідації розроблених математичних моделей, алгоритмів та програмних засобів було організовано серію комп'ютерних випробувань на реальному навчальному матеріалі курсів «Алгоритми та структури даних» та «Основи веброзробки».

Експериментальна вибірка формувалася протягом навчального семестру та включала 1200 анонімізованих сесій розв'язання практичних алгоритмічних задач здобувачами освіти спеціальності 122 «Комп'ютерні науки».

Структура експериментального набору даних:

- Загальна кількість здобувачів: 74 студенти академічних груп 3-го та 4-го курсів;
- Загальна кількість зафіксованих практичних спроб розв'язання (сабмітів): 1200 сесій;
- Кількість атомарних навичок у графі знань курсу (Knowledge Components): 18 навичок (включаючи лінійні масиви, стеки, черги, рекурсивні дерева, алгоритми сортування, базові веб-протоколи);
- Обсяг збережених квантів темпоральної телеметрії кодописання: 142 800 записів із середньою частотою 1 квант на 1.5 секунди.

Специфікація 18 атомарних навичок тестового графа курсу та їхні апріорні базові параметри калібрування BKT наведено в табл. 5.1.

Таблиця 5.1 — Специфікація навичок графа знань курсу та їхні апріорні параметри калібрування

Код навички	Назва компонента знань (KC)	Базова ініціалізація P(L_0)	Швидкість навчання P(T)	Апріорне вгадування P(G_0)	Апріорний промах P(S_0)
KC-01	Синтаксис масивів та індексація	0.40	0.25	0.15	0.10
KC-02	Двовимірні матриці та ітерація	0.25	0.20	0.12	0.12
KC-03	Стек (LIFO) та базові операції push/pop	0.20	0.18	0.10	0.10
KC-04	Черга (FIFO) та кільцевий буфер	0.20	0.18	0.10	0.10
KC-05	Однозв'язні динамічні списки	0.15	0.15	0.08	0.15
KC-06	Двозв'язні списки та вставка вузлів	0.15	0.15	0.08	0.15
KC-07	Базові сортування бульбашкою та вставками	0.30	0.22	0.15	0.10
KC-08	Швидке сортування QuickSort та розбиття Ломуто	0.10	0.12	0.05	0.18
KC-09	Сортування злиттям MergeSort та рекурсивне ділення	0.10	0.12	0.05	0.18
KC-10	Бінарний пошук у впорядкованому масиві	0.25	0.20	0.10	0.12
KC-11	Хеш-таблиці та колізії методом ланцюжків	0.15	0.15	0.08	0.15

Код навички	Назва компонента знань (KC)	Базова ініціалізація P(L_0)	Швидкість навчання P(T)	Апріорне вгадування P(G_0)	Апріорний промах P(S_0)
KC-12	Дерева бінарного пошуку (BST): вставка та пошук	0.10	0.14	0.06	0.16
KC-13	Балансування дерев: AVL-ротації	0.05	0.10	0.04	0.20
KC-14	Обхід графів у ширину (BFS)	0.12	0.15	0.08	0.14
KC-15	Обхід графів у глибину (DFS) та стек викликів	0.12	0.15	0.08	0.14
KC-16	Алгоритм Дейкстри пошуку найкоротшого шляху	0.08	0.11	0.05	0.18
KC-17	Динамічне програмування: мемоізація та одновимірні задачі	0.08	0.10	0.04	0.20
KC-18	Динамічне програмування: двовимірна оптимізація (Knapsack)	0.05	0.08	0.03	0.22.

Програма комп'ютерних експериментів передбачала перевірку трьох ключових гіпотез:

1. Гіпотеза метрологічної точності: впровадження динамічних регуляторів P(G_t) та P(S_t) на основі сенсорної телеметрії забезпечує статистично достовірне зниження середньоквадратичної похибки прогнозування результатів наступного кроку (RMSE) та зростання площі під ROC-кривою (AUC-ROC);

2. Гіпотеза стійкості до генеративного шахрайства: інтегральний коефіцієнт

довіри C_trust дозволяє виявляти понад 70% спроб недоброчесного використання стороннього згенерованого коду (ChatGPT, Copilot), запобігаючи хибному зарахуванню майстерності (False Mastery);

3. Гіпотеза масштабованості та економічної життєздатності: гексагональне WebSocket-ядро забезпечує затримку обробки телеметрії менше ніж 10 мс при одночасному навантаженні понад 500 сесій, а смарт-контракт SoulboundCertificate у мережі Polygon мінімізує транзакційні витрати газу до прийнятного для університетів рівня (< $0.01 за сертифікат).

Методика статистичної обробки даних включала розрахунок 95% довірчих інтервалів, перевірку нормальності розподілу за критерієм Шапіро-Уїлка та однофакторний дисперсійний аналіз (One-Way ANOVA) для оцінки статистичної значущості відмінностей між групами.

5.2. Емпіричне порівняння діагностичної точності моделей (RMSE, AUC-ROC, час збіжності)

5.2. Емпіричне порівняння діагностичної точності моделей (RMSE, AUC-ROC, час збіжності)

Для кількісної оцінки діагностичної спроможності порівнювалися три моделі:

1. Standard BKT — класична модель Корбетта-Андерсона зі статичними параметрами за замовчуванням (P(G) = 0.20, P(S) = 0.10, P(T) = 0.15, P(L_0) = 0.15);

2. 1PL IRT (Rasch Model) — одновимірна психометрична модель із попереднім калібруванням складності завдань за методом максимальної правдоподібності (MML);

3. Telemetry-Aware BKT — запропонована в роботі модифікована модель із динамічними регуляторами вгадування та промаху на базі сенсорної телеметрії та правилом Clamping (P_min = 0.05).

Для оцінки якості прогнозування використовувалися метрики:

1) Середньоквадратична похибка прогнозування (Root Mean Squared Error — RMSE):

$$RMSE = \sqrt{((1 / N) \sum_{i=1}^{N} (Y_i - P(Y_i = 1))^2)} \quad (5.1)$$

2) Середня абсолютна похибка (Mean Absolute Error — MAE):

$$MAE = (1 / N) \sum_{i=1}^{N} |Y_i - P(Y_i = 1)| \quad (5.2)$$

3) Площа під кривою помилок класифікації (Area Under the Receiver Operating Characteristic Curve — AUC-ROC).

4) Середній час обчислення одного кроку оновлення моделі t_calc.

Результати порівняльного експерименту на вибірці з 1200 сесій наведено в табл. 5.2.

Таблиця 5.2 — Порівняння метрологічних характеристик моделей діагностики знань

Модель оцінювання знань	**Похибка прогнозу RMSE**	**Похибка MAE**	**Метрика якості AUC-ROC**	**Середній час обчислення t_calc**
Standard BKT (Corbett-Anderson)	0.382	0.294	0.748	0.82 мс
1PL IRT (Rasch Model)	0.405	0.318	0.712	4.15 мс
Telemetry-Aware BKT (Запропонована)	0.214	0.162	0.871	1.14 мс.

Як демонструють дані табл. 5.2, запропонована модель забезпечує:

- Зниження середньоквадратичної похибки RMSE на 44.0% (з 0.382 до 0.214) порівняно з класичним BKT;
- Зниження середньої абсолютної похибки MAE на 44.9% (з 0.294 до 0.162);
- Підвищення площі під ROC-кривою з 0.748 до 0.871 (+16.4%), що свідчить про високу якість бінарної класифікації станів володіння навичками;

- Збереження субмілісекундної швидкості обчислень (1.14 мс на один крок), що у 3.6 раза швидше за чисельні ітерації моделі IRT.

Статистична значущість різниці за парним t-критерієм Стьюдента: $t = 8.42$ при $p < 0.001$, що відкидає нульову гіпотезу про випадковість покращення результатів.

5.3. Дослідження стійкості системи до академічного шахрайства та використання LLM-генераторів

5.3. Дослідження стійкості системи до академічного шахрайства та використання LLM-генераторів

Для експериментальної перевірки ефективності телеметричного захисту від недоброчесного використання стороннього генеративного ШІ (ChatGPT, Claude, GitHub Copilot) було проведено спеціалізоване випробування із залученням контрольної групи користувачів (200 тестових сесій).

Учасникам було поставлено завдання розв'язати серію завдань з алгоритмізації двома протилежними методами:

- Група А (сумлінне кодописання): написання алгоритму самостійно, з поступовим набором та виправленням помилок;
- Група Б (генеративне копіювання): швидке копіювання коду, згенерованого мовною моделлю за текстовою умовою завдання.

Фіксувалися показники:

1. Частка випадків безпідставного зарахування майстерності (False Mastery Rate — FMR): ситуація, коли учень, не володіючи навичкою, отримує статус майстерності $P(L) \geq 0.85$ завдяки вставці коду;
2. Середній інтегральний коефіцієнт академічної довіри сесії C_trust.

Результати порівняльного експерименту зведено в табл. 5.3.

Таблиця 5.3 — Результати тестування стійкості систем до генеративного копіювання

Режим оцінювання	Зафіксований рівень FMR	Середній індекс довіри C_trust	Статус сертифікації SBT
Класична LMS (Moodle / Canvas)	100.0% (усі зараховані)	Не вимірюється (індекс відсутній)	Доступ відкритий без обмежень
Standard BKT без телеметрії	86.5% (майже всі зараховані)	Не вимірюється (індекс відсутній)	Автоматично зараховано майстерність
Smart-BKT-Chain (Запропонована)	12.5% (зниження на 74.0%)	0.24 ± 0.06 (критично низький)	Блоковано (перенаправлено на рев'ю).

Аналіз табл. 5.3 доводить, що завдяки комплексному аналізу частки вставки R_paste > 0.60 та аномальної швидкості розв'язання ($\Delta t_solve < 5$ с) динамічний регулятор BktEngine своєчасно підвищує штрафне значення P(G_t) до 0.90, що запобігає неправомірному росту рейтингу P(L).

Розглянемо детальний кейс-стаді динаміки двох конкретних студентів на послідовності з 5 практичних завдань за навичкою КС-08 «Швидке сортування QuickSort» (табл. 5.4).

Таблиця 5.4 — Кейс-стаді порівняння динаміки рейтингу знань сумлінного студента та студента з LLM-копіюванням

Номер кроку	Студент А (самостійний): WPM / C_trust / P(L)	Студент Б (LLM-копіювання): WPM / C_trust / P(L)	Керуюче рішення BktEngine для Студента Б
Крок 1	38 WPM	C_trust = 0.88	P(L) = 0.28
Крок 2 (помилка)	42 WPM	C_trust = 0.85	P(L) = 0.21
Крок 3 (успіх)	46 WPM	C_trust = 0.89	P(L) = 0.44
Крок 4 (успіх)	51 WPM	C_trust = 0.91	P(L) = 0.72
Крок 5 (успіх)	48 WPM	C_trust = 0.92	P(L) = 0.89 (майстер)

Дані кейс-стаді демонструють фундаментальну відмінність: тоді як сумлінний учень А планомірно досягає статусу майстерності P(L)=0.89, учень Б навіть за умови 100% проходження тестів компілятора залишається на рівні

P(L)=0.19 через блокування вставленого коду регулятором P(G_t). Це унеможливлює отримання незаслуженого сертифіката.

5.4. Навантажувальне тестування веб-сокетного ядра та оптимізація транзакційних витрат у блокчейні

5.4. Навантажувальне тестування веб-сокетного ядра та оптимізація транзакційних витрат у блокчейні

Оцінка масштабованості та швидкісних характеристик платформи включала стрес-тестування бекенду та аналіз економічної ефективності смарт-контрактів.

1. Стрес-тестування гексагонального бекенду apps/api:

За допомогою утиліти навантажувального тестування було змодельовано одночасну активність 1000 віртуальних клієнтів, які транслювали пакети телеметрії через WebSocket-з'єднання з частотою 1 пакет на секунду.

Метрики продуктивності сервера:

- Середня затримка обробки телеметричного пакета в гейтвеї: 3.4 мс;
- 95-й процентиль затримки (P95 Latency): 6.8 мс;
- 99-й процентиль затримки (P99 Latency): 11.2 мс;
- Споживання оперативної пам'яті сервером: 280 МБ;
- Утилізація центрального процесора (CPU Load): 24% на 4 ядрах AMD Ryzen.

Отримані результати підтверджують високу готовність бекенду до обслуговування одночасних потокових лабораторних робіт у межах цілих факультетів.

2. Аналіз транзакційних витрат смарт-контракту SoulboundCertificate.sol:

Для оцінки витрат було проведено розгортання та тестове карбування 100 токенів сертифікатів у тестовій та основній мережах Polygon PoS (табл. 5.5).

Таблиця 5.5 — Показники транзакційних витрат у мережі Polygon PoS

Операція смарт-контракту	Витрати газу (Gas Units)	Вартість при 35 Gwei (MATIC)	Еквівалент у USD ($)
Розгортання контракту (Deploy)	1 428 500 gas	0.0500 MATIC	$0.0450 USD
Карбування сертифіката (Mint)	84 210 gas	0.0029 MATIC	$0.0026 USD
Оновлення статусу аудиту (Update)	32 150 gas	0.0011 MATIC	$0.0009 USD
Спалення / відкликання (Revoke)	28 400 gas	0.0009 MATIC	$0.0008 USD.

Як свідчать дані табл. 5.5, середня вартість випуску захищеного непередаваного сертифіката Soulbound Token становить менше ніж $0.003 USD (менше 15 копійок). Це усуває економічний бар'єр для масового практичного впровадження децентралізованої сертифікації в освітніх закладах будь-якого рівня.

ЗАГАЛЬНІ ВИСНОВКИ

У кваліфікаційній магістерській роботі успішно вирішено актуальне науково-прикладне завдання щодо підвищення ефективності індивідуалізованого навчання програмуванню та забезпечення високої достовірності оцінювання навчальних досягнень здобувачів освіти. Створено теоретико-ймовірнісну базу, алгоритмічний апарат та повнофункціональний програмний комплекс інтелектуальної EdTech-системи адаптивного навчання та децентралізованого оцінювання знань на основі моделі Bayesian Knowledge Tracing та технології блокчейн.

На основі проведених теоретичних досліджень, алгоритмічного проєктування, програмної реалізації та експериментальної валідації отримано такі основні наукові та практичні результати:

1. Здійснено порівняльний аналіз наявних систем електронного навчання (LMS Moodle/Canvas, ITS ALEKS/Knewton Alta та блокчейн-реєстрів). Визначено межі застосовності кожного підходу: орієнтація класичних LMS на статичний результат сабміту без контролю динаміки написання коду, потреба адаптивних ITS у розширенні контекстного дидактичного супроводу для попередження когнітивної втоми здобувачів та необхідність органічного зв'язку між блокчейн-сертифікатами і первинним процесом формування компетентностей.

2. Досліджено математичний апарат сучасних моделей відстеження знань (BKT, IRT, DKT). Формалізовано та проаналізовано граничні режими класичної моделі Корбетта-Андерсона («Lucky Guess Spike», «Zero-Learning Sink» та «The Frustration Loop»), які обумовлюють асимптотичне насичення та надмірну інертність оцінки чистого BKT в інтерактивному середовищі написання вихідного коду.

3. Удосконалено ймовірнісну модель Bayesian Knowledge Tracing завдяки введенню динамічних коригувальних регуляторів ймовірностей випадкового вгадування P(G_t) та промаху P(S_t), що розраховуються в режимі реального

часу на основі високочастотної темпоральної сенсорної телеметрії клієнтського редактора Monaco Editor (швидкості набору WPM, частки вставки з буфера R_paste, латентності осмислення завдання T_first_key та динаміки правок N_backspaces).

4. Впроваджено та теоретично обґрунтовано математичне правило числових границь з гарантованою асимптотою (Clamping Rule) з нижньою межею знань P_min = 0.05, що ліквідувало ефект обчислювальної втрати розрядів у середовищі V8 та забезпечило оптимальний темп реабілітації оцінки здобувача (за 3–4 успішні спроби замість понад 20) після подолання складнощів.

5. Розроблено та формалізовано алгоритм тактичного відступу за графом навчальних пререквізитів (DAG Backtracking) та трирівневу систему дидактичного супроводу (Remediation Scaffolding: Сократівські підказки, картки типових оман Misconception Cards, інтервали когнітивного розвантаження), що дозволило скоротити частку припинення спроб здобувачами освіти (Drop-out rate) з 42.0% до 5.8% на складних алгоритмічних темах.

6. Запропоновано науково-обґрунтовану гібридну дворівневу архітектуру децентралізованої сертифікації: високопродуктивне off-chain аналітичне ядро на базі NestJS та WebSocket-шлюзів для обробки потоку телеметрії із затримкою до 3.4 мс та децентралізований on-chain шар у блокчейн-мережі другого рівня Polygon PoS на базі непередаваних смарт-контрактів стандарту Soulbound Tokens (ERC-5192) із фіксацією криптографічного аудиторського хешу сесії H_audit.

7. Спроєктовано та реалізовано модульний повнофункціональний програмний комплекс Smart-BKT-Chain з чітким розподілом ролей між 5 застосунками монорепозиторію: простором первинної дидактики apps/study з рушієм фокусування Zen Focus Reading Engine та пісочницею CodePlayground, тренажером учня apps/student, кабінетом викладача apps/teacher з тепловою картою та чергою ReviewQueueInspector, CMS методиста apps/admin та бекендом apps/api.

8. Розроблено та впроваджено інженерну систему управління життєвим циклом розробки комплексу на базі гнучкої методології Agile та Atlassian Jira, що включає декомпозицію функціоналу за 10 архітектурними епіками, суворий регламент семантичного маркування коммітів (Jira Conventional Commits) із автоматизованою валідацією через Git Hooks та дворівневі критерії готовності коду (Definition of Done).

9. Проведено комплексне експериментальне дослідження на вибірці з 1200 реальних навчальних сесій, результати якого підтвердили високу ефективність розробленої системи:

- середньоквадратична похибка прогнозування знань RMSE знизилася на 44.0% (з 0.382 до 0.214) порівняно з класичним BKT;
- площа під ROC-кривою класифікації навичок (AUC-ROC) зросла з 0.748 до 0.871;
- кількість випадків безпідставного зарахування майстерності при копіюванні розв'язків з генеративного ШІ скоротилася на 74.0%;
- середня вартість емісії непередаваного сертифіката склала 84 210 одиниць газу (< $0.003 USD), що доводить технічну та економічну масштабованість платформи.